

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**
**Федеральное государственное бюджетное образовательное учреждение
высшего профессионального образования
«Вятский государственный университет»
(ФГБОУ ВПО «ВятГУ»)**
**Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин**

**Методические указания
к самостоятельным и лабораторным работам
по дисциплине «Базы данных»**

**Лабораторная работа №2_2.
Основы DML-запросов в PostgreSQL**

Цели лабораторной работы:

- научиться использованию агрегатных функций,
- научиться использованию операторов и встроенных функции, работе с датами,
- научиться использованию подзапросами и предложениями with, returning,
- рассмотреть и выполнить различные планы выполнения запросов,
- выполнить FORMAT JSON для выборки из нескольких связанных таблиц,
- выполнить три способа соединения для одной и той же выборки и определить наиболее оптимальный.

Выполнить все примеры для своей предметной области.

Использовать данные и знания, полученные в предыдущих лаб. работах

Агрегатные функции

Часто при выполнении запросов нужно получить результат выполнения какой-либо функции над множеством значений (сумма, среднее значение, подсчет количества, объединение в массив или строку (text) и т.д.). Для этого используются агрегатные функции.

В PostgreSQL существует большое количество различных агрегатных функций. *Подробнее про них написано в документации.*

Если необходимо получить результат выполнения агрегатной функции над разными группами строк из выборки, необходимо использовать предложение group by. На основе совпадения значений в указанных столбцах строки объединяются в группы.

В таблице 1 перечислены некоторые агрегатные функции общего назначения.

Таблица 1 – Некоторые агрегатные функции общего назначения

Функция	Описание
avg(<i>выражение</i>)	арифметическое среднее для всех входных значений
max(<i>выражение</i>)	максимальное значение <i>выражения</i> среди всех входных данных
min(<i>выражение</i>)	минимальное значение <i>выражения</i> среди всех входных данных
string_agg(<i>выражение</i> , <i>разделитель</i>)	входные данные складываются в строку через заданный разделитель

<code>sum(выражение)</code>	сумма значений <i>выражения</i> по всем входным данным
<code>count(*)</code>	количество входных строк
<code>count(выражение)</code>	количество входных строк, для которых значение <i>выражения</i> не равно NULL

-- Пример 1

-- Два запроса ниже эквиваленты и возвращают количество записей в *student*

```
select count(*) from student;
select count(1) from student;
```

count
5

-- Пример 2

-- Возвращает строку с четырьмя значениями: сумма по столбцу *number_of_hours*, минимальное, максимальное и среднее значения в *number_of_hours*

```
select sum(number_of_hours), min(number_of_hours), max(number_of_hours),
avg(number_of_hours) from subject;
```

sum	min	max	avg
54	18	36	27

-- Пример 3

Для выбора данных по каждому студенту нужно использовать группировку.

Правило – в разделе *select* могут быть только поля группировки и агрегатные функции.

-- Этот запрос выбирает всех студентов и сданные ими предметы, отображает:
-- *subjects* – список предметов (имена, разделенные запятой)
-- *average_rating* – средний балл
-- Записи отсортированы по возрастанию по среднему баллу
-- Важно отметить, что все столбцы *st.id*, *st.course* нужно указать в *group by*, чтобы выполнить группировку, поскольку они указаны в *select*

```
select st.id, st.course,
string_agg(sb.name, ', ') as subjects, avg(p.rating) as average_rating
from student st, subject sb, progress p
where p.id_subject = sb.id and p.id_student = st.id
group by st.id, st.course order by average_rating asc;
```

id	course	subjects	average_rating
3	3	История	3
1	3	Технологии программирования	4
2	3	Технологии программирования, История	4,5
4	4	История	5

Предложение *having*

Иногда возникает потребность сделать выборку, в которой строки отвечают определённому условию, вычисляемому с помощью агрегатных функций.

Такие условия следует записывать не в предложение `where`, а в предложение `having`. Оно отличается от `where`: `where` фильтрует отдельные строки до применения `group by`, а `having` фильтрует строки групп, созданных предложением `group by` или когда в условии используется агрегатная функция.

-- Пример 4

-- Этот запрос вернет записи, у которых средний балл больше 4

```
select st.id, st.course,
string_agg(sb.name, ', ') as subjects, avg(p.rating) as average_rating
from student st, subject sb, progress p
where p.id_subject = sb.id and p.id_student = st.id
group by st.id, st.course having avg(p.rating) > 4
order by average_rating asc;
```

id	course	subjects	average_rating
2	3	Технологии программирования, История	4,5
4	4	История	5

Операторы и встроенные функции, работа с датами

Во многих примерах ранее использовались арифметические и логические операторы, а также различные встроенные функции (например, `now()`, возвращающая дату начала текущей транзакции).

PostgreSQL предоставляет большое количество операторов и встроенных функций. *Подробнее можно прочитать в документации.*

Поскольку их количество очень велико, а написание совпадает во многих случаях с общепринятым в языках программирования, здесь не будет приводиться списков операторов и встроенных функций. Вместо этого ниже показаны примеры различных часто встречаемых или неочевидных ситуаций, которые могут связанных с их использованием.

-- Пример 5

-- Есть три логических оператора: `and`, `or`, `not`. Приоритет стандартный

```
select not 1 < 2 and 3 < 4 or 5 = 5; -- true
```

-- Пример 6

-- Оператор "не равно" можно записывать как в виде `<>`, так и `!=`

```
select 1 <> 1 or 5 != 5; -- false
```

-- Пример 7

-- Нельзя сравнивать значение с `null` с помощью `=` или `<>`. Нужно использовать предикаты `is null`, `is not null`.

```
select 1 is null or 5 is not null; -- true
```

-- Пример 8

-- Для проверки вхождения в промежуток есть предикаты *between*, *now between*
`select 3 between 3 and 5; -- true`
`select 1 not between -1 and 5; -- false`

-- Пример 9

-- Для конкатенации строк используется оператор `||` или функция *concat*
`select 'Hello, ' || 'Postgre!'; -- Hello, Postgre!`
`select concat('Hello, ', 'Postgre!'); -- Hello, Postgre!`

-- Пример 10

-- Разница между ними в том, что строка `|| null = null`, а функция *concat* воспринимает *null*, как пустую строку
`select 'abc' || null; -- null`
`select concat('abc', null, 'de'); -- abcde`

-- Пример 11

-- Чтобы вставить перенос строки, можно сделать так. Функция *chr* возвращает символ по его коду в кодировке БД
`select ' first' || chr(13) || 'second'; -- first \n second`

-- Пример 12

-- Но если нужно поставить одинарную кавычку в строке, можно поставить одинарных кавычки подряд
`select 'Don't worry'; -- Don't worry`

-- Пример 13

-- Для приведения в верхний или нижний регистр используются функции *lower*, *upper*
`select lower('AbC'); -- abc`
`select upper('AbC'); -- ABC`

-- Пример 14

-- Функция *trim* обрезает пробельные символы по бокам строки
`select trim(' a b c '); -- "a b c"`

-- Пример 15

-- Конструкцию *case* можно использовать по аналогии со *switch* или для выборки разных значений по условию
`select case when 1 = 1 then 'AAA' else 'OOO' end; -- AAA`

-- Пример 16

-- Полезная функция *coalesce(a, b, c, ...)*, которая вернет первый аргумент, отличный от *null*
`select coalesce(null, null, 42, null, 33); -- 42`

-- Пример 17

-- Строки возможно проверять по шаблону с помощью оператора *like*. В шаблоне `%` означает любое количество символов, `_` – один символ
`select 'abcdef' like '%cdef'; -- true`
`select 'abcdef' like 'a_cdef'; -- true`
`select 'abcdef' like '%c%'; -- true`
`select 'abcdef' like 'ab_'; -- false`

-- Пример 18

-- Часто преобразование типов происходит неявно
`select 42 || '...'; -- "42..."`

-- Пример 19

-- Но можно делать его вручную с помощью оператора `::`
`select 42::text; -- "42"`

```
-- Пример 20
-- Возвращает timestamp на начало текущей транзакции
select now();
```

В двух последних примерах было рассмотрено преобразование типов. Часто оно используется при написании дат в запросах (преобразовать строку в дату). В этом случае принимается практически любой стандартный формат даты. Тем не менее, он во многом определяется местоположением клиента, и, чтобы избежать неожиданных эффектов, лучше использовать функции `to_date`, `to_timestamp`, принимающих вторым аргументом формат даты. Примеры показаны ниже.

```
-- Пример 21
-- Два запроса ниже аналогичны, если формат даты на клиенте совпадает с
указанным
select '12-05-2019'::date;
select to_date('12-05-2019', 'DD-MM-YYYY');
```

```
-- Пример 22
-- Два запроса ниже аналогичны, если формат даты на клиенте совпадает с
указанным
select '12-05-2019 12:08'::timestamp;
select to_timestamp('12-05-2019 12:08', 'DD-MM-YYYY HH:MI');
```

Если суммировать `date` и число, к дате прибавится указанное количество дней. Суммировать `timestamp` и число нельзя. Для более очевидного прибавления временных промежутков лучше суммирование не с числами, а с временными интервалами. Важно помнить, что при сложении `date` и временного интервала получается `timestamp`. Примеры показаны ниже.

```
-- Пример 23
-- Прибавляет один день к дате
select to_date('12-05-2019', 'DD-MM-YYYY') + 1; -- 13-05-2019
```

```
-- Пример 24
-- Складывать timestamp и число нельзя
select to_timestamp('12-05-2019 12:08', 'DD-MM-YYYY HH:MI') + 1; -- Ошибка
```

```
-- Пример 25
-- К датам лучше прибавлять временные интервалы
select to_date('12-05-2019', 'DD-MM-YYYY') + interval '1 day'; -- 13-05-2019
00:00
select to_date('12-05-2019', 'DD-MM-YYYY') - interval '5 year'; -- 12-05-2014
00:00
select to_timestamp('12-05-2019 12:08', 'DD-MM-YYYY HH:MI') + interval '1
year 2 month 11 second'; -- 12-07-2020 00:08:11
```

```
-- Пример 26
-- Даты также можно вычитать, результат - количество дней
select to_date('12-05-2019', 'DD-MM-YYYY') - to_date('10-05-2019', 'DD-MM-
YYYY'); -- 2
```

-- Пример 27

-- Если вычесть date из timestamp, timestamp из date или timestamp из timestamp, получится временной интервал
select to_timestamp('12-05-2019 12:08', 'DD-MM-YYYY HH:MI') -
to_date('10-05-2019', 'DD-MM-YYYY');
-- 0 years 0 mons 2 days 0 hours 8 mins 0.00 secs

Подзапросы и предложение with

Если запрос что-либо возвращает, к результатам его работы можно обратиться в другом запросе. Например, в разделе from помимо таблиц можно указывать подзапросы (при этом обязательно давать им имена).

Примеры использования подзапросов показаны ниже.

-- Пример 28

-- Бесполезный пример

select * **from** (**select** * **from** student) s;

id	name	surname	middle_name	id_group	address	course
1	Николай	Котов	Сергеевич	4	г. Киров ул. Ломоносова 16а	3
2	Василий	Олоев	Петрович	4	г. Киров ул. Свободы 4	3
3	Олег	Зотов	Георгиевич	5	г. Киров ул. Воровского 122	3
4	Константин	Романов	Владимирович	6	г. Киров ул. Ленина 52	4

-- Пример 29

-- Запрос может обращаться к полям вложенного запроса не глубже, чем на один уровень

-- Вложенные запросы одного уровня не могут обращаться друг к другу

select s.* **from** (-- **select** s.* здесь сделать нельзя

select ss.* **from** (**select** * **from** student) ss -- Дописать на этом же уровне еще один вложенный запрос, который обращается к ss, нельзя
) s;

-- Пример 30

-- Если возвращается одна строка, то подзапросы можно использовать так (с оператором =)

delete from student **where** id = (**select** id_student **from** progress **where** rating = '5');

-- Пример 31

-- Если значений несколько, можно использовать оператор in

select name, surname **from** student **where** id **in** (**select** id_student **from** progress **where** rating = '4');

name	surname
Николай	Котов
Василий	Олоев

Использование подзапросов очень полезно и многие задачи нельзя решить без их применения. Тем не менее, с этим связаны некоторые проблем:

- область видимости результатов подзапроса ограничена;
- если для выполнения всего запроса необходимо СУБД обратиться много раз к результатам подзапроса, он может выполняться каждый раз.

Указанные выше проблемы решаются с помощью предложения `with`. Оно может размещаться перед командами `select`, `insert`, `update` и `delete`. В нем можно расположить несколько именованных подзапросов и далее использовать их результаты. Такие подзапросы называют *общими табличными выражениями* (Common Table Expressions, CTE).

Каждое CTE может обращаться к результатам работы вышестоящих CTE в предложении `with`. Создаваемая в этом случае выборка вычисляется только один раз и сохраняется во временную таблицу, существующую на время выполнения всего запроса. Кроме того, запросы с предложениями `with` гораздо проще читать и понимать.

Подробнее о предложении `with` и CTE можно прочитать в документации.

Ниже показаны примеры использования предложения `with`.

-- Пример 32

--Создание таблицы `good_students`

```
create table public.good_students(
    id bigserial primary key,
    name varchar(30) not null,
    surname varchar(30) not null,
    middle_name varchar(30),
    id_group bigint not null references public.group(id),
    address varchar(100),
    course int check (course >= 1 and course <=5)
);
```

-- Этот запрос перенесет записи из таблицы `student` в таблицу `good_students` данные студентов с оценками > 4

```
with items_to_insert as (select id_student from progress where rating < 4)
insert into good_students (select id, name, surname, middle_name, id_group,
address, course from student where id in (select * from items_to_insert));
```

Предложение `returning`

Полезным часто бывает то, что команды `insert`, `update` и `delete` могут возвращать строки. Для этого используется предложение `returning`, которое ставится в конце запроса и возвращает строки, которые были затронуты выполнением команды. Для `update` возвращаются уже измененные значения строк.

Результат, получаемый с помощью `returning`, нельзя использовать как подзапрос, но можно использовать в предложении `with`. С помощью этого,

например, просто сохранять ссылочную целостность при заполнении таблиц данными: в предложении `with` можно вставлять строки, на которые будут ссылаться вставляемые в основной части запроса.

Ниже показаны примеры использования предложения `returning`.

-- Пример 33

-- Этот запрос обновит таблицу `subject` и вернет все обновленные строки

```
update subject set number_of_hours = number_of_hours - 2 where
number_of_hours >= 30 returning *;
```

id	name	count_of_hours
2	История	34

Создать и выполнить SQL-скрипты для выполнения различных планов запросов.

Часто необходимо проверить производительность только что написанного запроса в PostgreSQL в поисках способа улучшить его производительность. Для этого вам нужно получить отчет о выполнении запроса, который называется планом выполнения. План выполнения запроса дает суммарную информацию о выполнении запроса с подробным отчетом о времени, потраченном на каждом шаге, и затратах на его выполнение.

Структура плана выполнения запроса

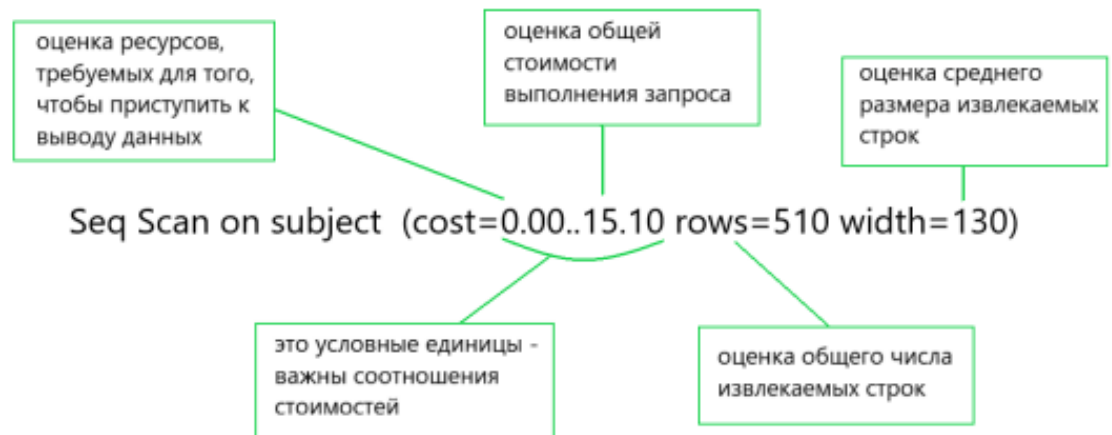
Структура плана запроса представляет собой дерево, состоящее из так называемых узлов плана (`plan nodes`).

Узлы на нижних уровнях дерева отвечают за просмотр и выдачу строк таблиц, которые осуществляются с помощью методов доступа, описанных выше. Если конкретный запрос требует выполнения операций агрегирования, соединения таблиц, сортировки, то над узлами выборки строк будут располагаться дополнительные узлы дерева плана.

Например, для соединения наборов строк будут использоваться способы, которые мы только что рассмотрели.

Для каждого узла дерева плана команда `EXPLAIN` выводит по одной строке, при этом выводятся также оценки стоимости выполнения операций на каждом узле, которые делает планировщик. В случае необходимости для конкретных узлов могут выводиться дополнительные строки. Самая первая строка плана содержит общую оценку стоимости выполнения данного запроса.

--Пример 71 `EXPLAIN select * from subject;`



Поскольку в этом запросе нет предложения WHERE, он должен просмотреть все строки таблицы, поэтому планировщик выбирает последовательный просмотр (sequential scan). В скобках приведены важные параметры плана.

Первая оценка равна нулю, поскольку никакие дополнительные операции с выбранными строками не предполагаются, и PostgreSQL может сразу же выводить прочитанные строки. Формируя эту оценку, планировщик исходит из предположения, что данный узел плана запроса выполняется до конца, т. е. извлекаются все имеющиеся строки таблицы. Исключения: в запросе SELECT предложение LIMIT. Обе оценки вычисляются на основе ряда параметров сервера баз данных.

Для каждого запроса планировщик формирует несколько планов. При сравнении различных вариантов плана, как правило, для выполнения выбирается тот, который имеет наименьшую общую стоимость выполнения запроса. Оценки числа строк и их размера планировщик получает на основе статистики, накапливаемой в специальных системных таблицах.

EXPLAIN

Сгенерировать план выполнения запроса позволяет ключевое слово EXPLAIN в PostgreSQL. Синтаксис создания плана в PostgreSQL имеет вид:

EXPLAIN [(параметр [, ...])] оператор

EXPLAIN [ANALYZE] [VERBOSE] оператор

Здесь допускается параметр:

- ANALYZE [boolean]
- VERBOSE [boolean]

- COSTS [boolean]
- BUFFERS [boolean]
- TIMING [boolean]
- FORMAT { TEXT | XML | JSON | YAML }

Параметры:

ANALYZE

Выполнить команду и вывести фактическое время выполнения и другую статистику. По умолчанию этот параметр равен FALSE.

--Пример 72 EXPLAIN (analyze) select * from student;

	QUERY PLAN
	text
1	Seq Scan on student (cost=0.00..11.60 rows=160 width=472) (actual time=0.032..0.034 rows=6 loops=1)
2	Planning Time: 0.251 ms
3	Execution Time: 0.067 ms

VERBOSE

Вывести дополнительную информацию о плане запроса. В частности, включить список столбцов результата для каждого узла в дереве плана, дополнить схемой имена таблиц и функций, всегда указывать для переменных в выражениях псевдоним их таблицы, а также выводить имена всех триггеров, для которых выдаётся статистика. По умолчанию этот параметр равен FALSE.

--Пример 73 EXPLAIN (verbose) select * from student;

	QUERY PLAN
	text
1	Seq Scan on public.student (cost=0.00..11.60 rows=160 width=472)
2	[...] Output: id, first_name, second_name, middle_name, id_group, address, course

COSTS

Вывести рассчитанную стоимость запуска и общую стоимость каждого узла плана, а также рассчитанное число строк и ширину каждой строки. Этот параметр по умолчанию равен TRUE.

--Пример 74 EXPLAIN (costs) select * from student;

QUERY PLAN	
	text
1	Seq Scan on student (cost=0.00..11.60 rows=160 width=472)

BUFFERS

Включить информацию об использовании буфера. В частности, вывести число попаданий, блоков прочитанных, загрязненных и записанных в разделяемом и локальном буфере, а также число прочитанных и записанных временных блоков. *Попаданием (hit)* считается ситуация, когда требуемый блок уже находится в кеше и чтения с диска удаётся избежать. Блоки в общем буфере содержат данные обычных таблиц и индексов, в локальном — данные временных таблиц и индексов, а временные блоки предназначены для краткосрочного использования при выполнении сортировки, хеширования, материализации и подобных узлов плана. Число *загрязнённых блоков (dirty)* показывает, сколько ранее не модифицированных блоков изменила данная операция; тогда как число *записанных блоков (written)* показывает, сколько ранее загрязнённых блоков данный серверный процесс вынес из кеша при обработке запроса. Значения, указываемые для узла верхнего уровня, включают значения всех его дочерних узлов. В текстовом формате выводятся только ненулевые значения. Этот параметр действует только в режиме ANALYZE. По умолчанию его значение равно FALSE.

--Пример 75 EXPLAIN (analyze,buffers) select * from student;

QUERY PLAN	
	text
1	Seq Scan on student (cost=0.00..11.60 rows=160 width=472) (actual time=2.137..2.140 rows=6 loops=1)
2	[...] Buffers: shared hit=1
3	Planning Time: 0.686 ms
4	Execution Time: 2.217 ms

TIMING

Включить в вывод фактическое время запуска и время, затраченное на каждый узел. Постоянное чтение системных часов может значительно замедлить запрос, так что если достаточно знать фактическое число строк, имеет смысл сделать этот параметр равным FALSE. Время выполнения всего оператора замеряется всегда, даже когда этот параметр выключен и на уровне

узлов время не подсчитывается. Этот параметр действует только в режиме ANALYZE. По умолчанию его значение равно TRUE.

--Пример 76 EXPLAIN (analyze,timing) select * from student;

QUERY PLAN	
	text
1	Seq Scan on student (cost=0.00..11.60 rows=160 width=472) (actual time=0.013..0.015 rows=6 loops=1)
2	Planning Time: 0.066 ms
3	Execution Time: 0.026 ms

FORMAT

Установить один из следующих форматов вывода: TEXT, XML, JSON или YAML. Последние три формата содержат ту же информацию, что и текстовый, но больше подходят для программного разбора. По умолчанию выбирается формат TEXT(выше приведены примеры использования формата TEXT). В дальнейшем все примеры будут рассмотрены в JSON формате представления запросов.

--Пример 77

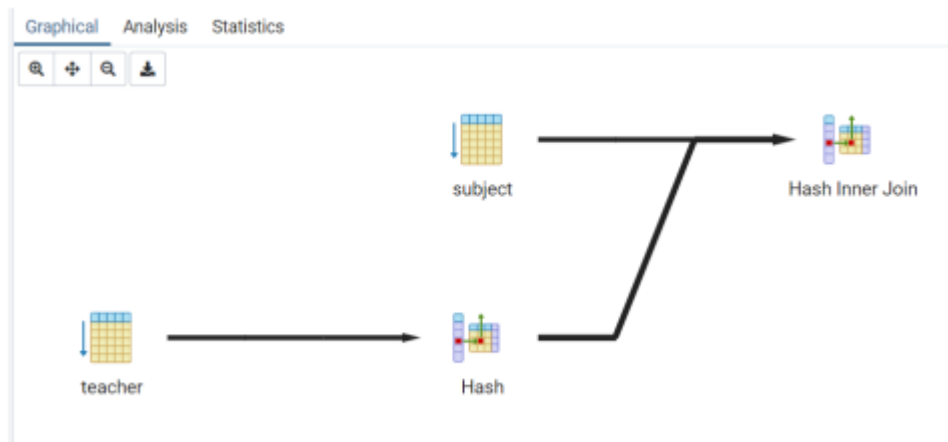
--Присоединение таблицы с id учителя к id предмета

EXPLAIN (FORMAT XML) select * from subject join teacher on subject.id = teacher.id;

```
<explain xmlns="http://www.postgresql.org/2009/explain">
  <Query>
    <Plan>
      <Node-Type>Hash Join</Node-Type>
      <Parallel-Aware>>false</Parallel-Aware>
      <Async-Capable>>false</Async-Capable>
      <Join-Type>Inner</Join-Type>
      <Startup-Cost>15.18</Startup-Cost>
      <Total-Cost>31.62</Total-Cost>
      <Plan-Rows>230</Plan-Rows>
      <Plan-Width>454</Plan-Width>
      <Inner-Unique>true</Inner-Unique>
      <Hash-Cond>(subject.id = teacher.id)</Hash-Cond>
    <Plans>
      <Plan>
        <Node-Type>Seq Scan</Node-Type>
```

✓ OK

--Пример 78 EXPLAIN (FORMAT JSON) select * from subject
join teacher on subject.id = teacher.id;



--Пример 79
EXPLAIN (FORMAT YAML) select * from subject
join teacher on subject.id = teacher.id;



Boolean

Включает или отключает заданный параметр. Для включения параметра можно написать TRUE, ON или 1, а для отключения — FALSE, OFF или 0. Значение boolean можно опустить, в этом случае подразумевается TRUE.

--Пример 80
EXPLAIN (timing off) select * from student;

QUERY PLAN	
	text
1	Seq Scan on student (cost=0.00..11.60 rows=160 width=472)

Оператор

Любой оператор SELECT, INSERT, UPDATE, DELETE, VALUES, EXECUTE, DECLARE, CREATE TABLE AS и CREATE MATERIALIZED VIEW AS, план выполнения которого вас интересует.

--Пример 81

--Добавление группы

EXPLAIN insert into public.group(name, number_of_students) values ('ИКТб-2301-01-00', 23);

QUERY PLAN	
	text
1	Insert on "group" (cost=0.00..0.01 rows=0 width=0)
2	[...] -> Result (cost=0.00..0.01 rows=1 width=90)

Методы доступа

Метод доступа характеризует тот способ, который используется для просмотра таблиц и извлечения только тех строк, которые соответствуют критерию отбора. Существуют различные методы доступа: те, при которых индекс не используется, и группа методов, основанных на использовании индекса. Поскольку и таблицы, и индексы хранятся на диске, то для работы с ними эти объекты считываются в память, в которой они представлены разбитыми на отдельные фрагменты, называемые страницами. Эти страницы имеют специальную структуру. Размер страниц по умолчанию составляет 8 килобайт.

Sequential scan(последовательный просмотр)

При выполнении последовательного просмотра (sequential scan) обращения к индексам не происходит, а строки извлекаются из табличных страниц в соответствии с критерием отбора. В том случае, когда в запросе нет предложения WHERE, тогда извлекаются все строки таблицы.

Данный метод применяется, когда требуется выбрать все строки таблицы или значительную их часть, т. е. когда так называемая селективность выборки низка. В таком случае обращение к индексу не ускорит процесс просмотра, а возможно даже и замедлит.

--Пример 82

EXPLAIN (FORMAT JSON) select * from subject;

Graphical Analysis Statistics		
subject		
subject		
Node Type	Seq Scan	
Parallel Aware	false	
Async Capable	false	
Relation Name	subject	
Alias	subject	
Startup Cost	0	
Total Cost	15.1	
Plan Rows	510	
Plan Width	130	
Loops	1	
_serial	1	

Index scan(просмотр на основе индекса)

Просмотр на основе индекса (*index scan*) предполагает обращение к индексу, созданному для данной таблицы. Поскольку в индексе для каждого ключевого значения содержатся уникальные идентификаторы строк в таблицах, то после отыскания в индексе нужного ключа производится обращение к соответствующей странице таблицы и извлечение искомой строки по ее идентификатору.

При этом нужно учитывать, что хотя записи в индексе упорядочены, но обращения к страницам таблицы происходят хаотически, поскольку строки в таблицах не упорядочены.

В таком случае при низкой селективности выборки, т. е. когда из таблицы отбирается значительное число строк, использование индексного поиска может не только не давать ускорения работы, но даже и снижать производительность.

--Пример 83

EXPLAIN (FORMAT JSON) select * from subject where subject.id =1;

Graphical Analysis Statistics		
subject_pkey		
subject_pkey		
Node Type	Index Scan	
Parallel Aware	false	
Async Capable	false	
Scan Direction	Forward	
Index Name	subject_pkey	
Relation Name	subject	
Alias	subject	
Startup Cost	0.15	
Total Cost	8.17	
Plan Rows	1	
Plan Width	130	
Index Cond	(id = 1)	
Loops	1	
_serial	1	

Index only scan(просмотр исключительно на основе индекса)

Просмотр исключительно на основе индекса (*index only scan*), как следует из названия метода, не должен, казалось бы, требовать обращения к строкам таблицы, поскольку все данные, которые нужно получить с помощью запроса, в этом случае присутствуют в индексе.

Однако в индексе нет информации о видимости строк транзакциям — нельзя быть уверенным, что данные, полученные из индекса, видны текущей транзакции. Поэтому сначала выполняется обращение к карте видимости (*visibility map*), которая существует для каждой таблицы. В ней одним битом отмечены страницы, на которых содержатся только те версии строк, которые видны всем без исключения транзакциям.

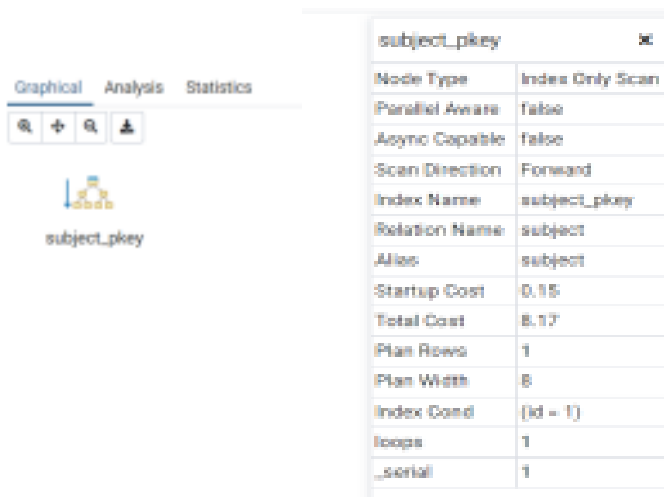
Если полученная из индекса версия строки находится на такой странице, значит, эта строка видна текущей транзакции и обращаться к самой таблице не требуется. Поскольку размер карты видимости очень мал, то в результате сокращается объем операций ввода/вывода.

Если же строка находится на странице, не отмеченной в карте видимости, тогда происходит обращение и к таблице; в результате никакого выигрыша по быстродействию в сравнении с обычным индексным поиском не достигается.

Просмотр исключительно на основе индекса особенно эффективен, когда выбираемые данные изменяются редко. Он может применяться, когда в предложении `SELECT` указаны только имена столбцов, по которым создан индекс.

--Пример 84

`EXPLAIN (FORMAT JSON) select subject.id from subject where subject.id =1;`



subject_pkey	
Node Type	Index Only Scan
Parallel Aware	false
Async Capable	false
Scan Direction	Forward
Index Name	subject_pkey
Relation Name	subject
Alias	subject
Startup Cost	0.15
Total Cost	8.17
Plan Rows	1
Plan Width	8
Index Cond	(id = 1)
Loops	1
_serial	1

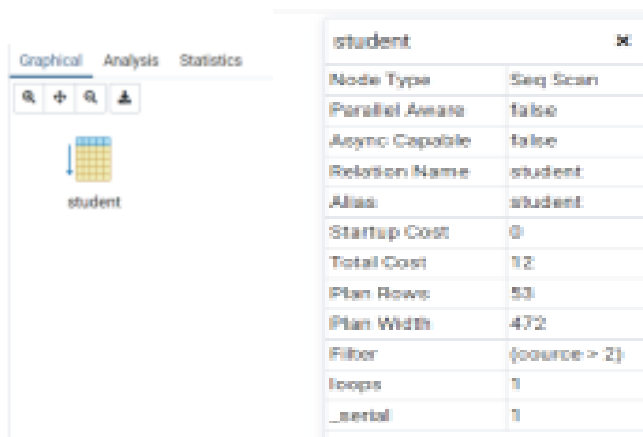
Bitmap scan(Просмотр на основе битовой карты)

Просмотр на основе битовой карты (bitmap scan) является модификацией просмотра на основе индекса.

Данный метод позволяет оптимизировать индексный поиск за счет того, что сначала производится поиск в индексе для всех искомых строк и формирование так называемой битовой карты, в которой указывается, в каких страницах таблицы эти строки содержатся. После того как битовая карта сформирована, выполняется извлечение строк из страниц таблицы, но при этом обращение к каждой странице производится только один раз.

--Пример 85

EXPLAIN (FORMAT JSON) select * from student where student.course > 2;



student	
Node Type	Seq Scan
Parallel Aware	false
Async Capable	false
Relation Name	student
Alias	student
Startup Cost	0
Total Cost	12
Plan Rows	53
Plan Width	472
Filter	{course > 2}
Loops	1
_serial	1

Способы соединения наборов строк

Другим важным понятием является способ соединения наборов строк (join). Набор строк может быть получен из таблицы с помощью одного из методов доступа, описанных выше.

Набор строк может быть получен не только из одной таблицы, а может быть результатом соединения других наборов. Важно различать способ соединения таблиц (JOIN) и способ соединения наборов строк.

Первое понятие относится к языку SQL и является высокоуровневым, логическим, оно не касается вопросов реализации. А второе относится именно к реализации, это — механизм непосредственного выполнения соединения наборов строк. Принципиально важным является то, что за один раз соединяются только два набора строк.

Существует три способа соединения: **вложенный цикл** (nested loop), **хеширование** (hash join) и **слияние** (merge join). Они имеют свои особенности, которые PostgreSQL учитывает при выполнении конкретных запросов.

Nested loop(вложенный цикл)

Суть способа «вложенный цикл» в том, что перебираются строки из «внешнего» набора и для каждой из них выполняется поиск соответствующих строк во «внутреннем» наборе. Если соответствующие строки найдены, то выполняется их соединение со строкой из «внешнего» набора. При этом способы выбора строк из обоих наборов могут быть различными.

Метод поддерживает соединения как на основе равенства значений атрибутов (эквисоединения), так и любые другие виды условий. Поскольку он не требует подготовительных действий, то способен быстро приступить к непосредственной выдаче результата.

Метод эффективен для небольших выборок.

--Пример 86

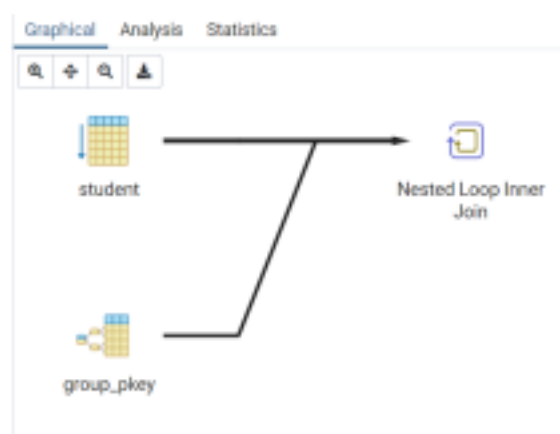
--Соединение таблиц по Id

set enable_mergejoin = off;

set enable_hashjoin = off;

EXPLAIN (FORMAT JSON)

select * from student join public.group on student.id_group = public.group.id



Nested Loop Inner Join		✖
Node Type	Nested Loop	
Parallel Aware	false	
Async Capable	false	
Join Type	Inner	
Startup Cost	0.15	
Total Cost	86.4	
Plan Rows	160	
Plan Width	562	
Inner Unique	true	
loops	1	
_serial	1	

Hash join(Соединение хешированием)

При соединении хешированием строки одного набора помещаются в хеш-таблицу, содержащуюся в памяти, а строки из второго набора перебираются, и для каждой из них проверяется наличие соответствующих строк в хеш-таблице.

Ключом хеш-таблицы является тот столбец, по которому выполняется соединение наборов строк. Как правило, число строк в том наборе, на основе которого строится хеш-таблица, меньше, чем во втором наборе. Это позволяет уменьшить ее размер и ускорить процесс обращения к ней.

Данный метод работает только при выполнении эквисоединений, поскольку для хеш-таблицы имеет смысл только проверка на равенство проверяемого значения одному из ее ключей.

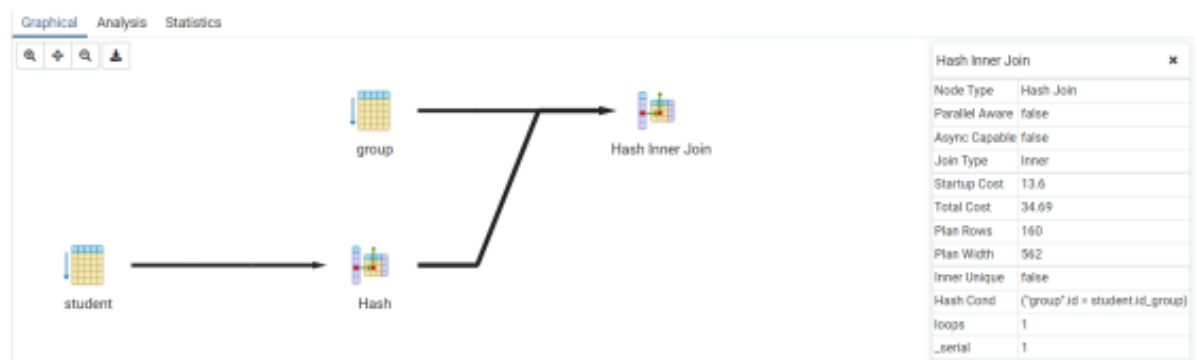
Метод эффективен для больших выборок.

--Пример 87

```
set enable_hashjoin = on;
```

```
EXPLAIN (FORMAT JSON)
```

```
select * from student join public.group on student.id_group = public.group.id
```



Merge join(Соединение методом слияния)

Соединение методом слияния производится аналогично сортировке слиянием. В этом случае оба набора строк должны быть предварительно отсортированы по тем столбцам, по которым производится соединение. Затем параллельно читаются строки из обоих наборов и сравниваются значения столбцов, по которым производится соединение. При совпадении значений формируется результирующая строка. Этот процесс продолжается до исчерпания строк в обоих наборах.

Этот метод, как и метод соединения хешированием, работает только при выполнении эквисоединений.

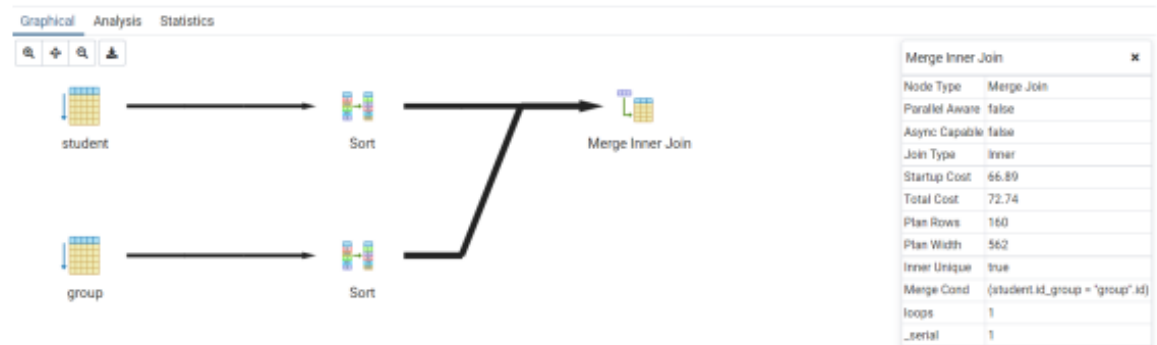
Он пригоден для работы с большими наборами строк.

--Пример 88

```
set enable_mergejoin = on;
```

```
set enable_hashjoin = off; EXPLAIN (FORMAT JSON)
```

```
select * from student join public.group on student.id_group = public.group.id
```



Управление планировщиком

Чтобы запретить планировщику использовать метод соединения на основе хеширования, нужно сделать так:

```
SET enable_hashjoin = off;
```

Чтобы запретить планировщику использовать метод соединения слиянием, нужно сделать так: SET enable_mergejoin = off;

А для того чтобы запретить планировщику использовать соединение методом вложенного цикла, нужно сделать так: SET enable_nestloop = off;

По умолчанию все эти параметры имеют значение «on» (включено).